

Softwarekonstruktion WS 16/17 – Übung 3

Ausgabe: 21.11.2016, Abgabe: 05.12.2016 12:00 Uhr

Präsenzübung

3.1 Grundlagen Hoare Kalkül

1. Betrachten Sie folgendes Programm **S**:

```
1 function func(x:Z, y:Z): Z
2   var a:Z
3     b:Z
4   begin
5     a := x
6     b := 0
7
8     while (a>0) do
9       begin
10        b := b+y
11        a := a-1
12      end
13
14   func := b
15 end
```

Beweisen Sie das Tripel: $\{x \geq 0\} \mathbf{S} \{func = x * y\}$
Verwenden sie als Hilfe das Programm HAHA¹.

Lösungsvorschlag

Test

```
1 function func(x:Z, y:Z): Z
2   precondition x>=0
3   postcondition func = x*y
4
5   var a:Z
6     b:Z
7   begin
8     a := x
9     {x>=0 /\ a=x}
10    b := 0
11    {x>=0 /\ a=x /\ b=0}
12    while (a>0) do
```

¹HAHA: <http://haha.mimuw.edu.pl/>

```

13 invariant a>=0 /\ b=(x-a)*y /\ x>=0
14 begin
15   b := b+y
16   {b-y = (x-a)*y /\ a>0 /\ x>=0}
17   a := a-1
18 end
19 {b=(x-a)*y /\ a=0 /\ x>=0}
20 func := b
21 end

```

2. Betrachten Sie folgendes Programmfragment S' in Pseudocode:

```

1 if x >= y
2   then
3     a := x - y
4   else
5     a := y - x
6 end

```

Beweisen Sie mittels Hoare Kalkül: $\{true\}\{S'\}\{a \geq 0\}$

Lösungsvorschlag

```

1 {} // -> Auswahlregel
2 if x>=y then
3   {x>=y}
4   a := x-y // -> Zuweisungsaxiom
5   {a>=0}
6 else
7   {x<y}
8   a := y-x // -> Zuweisungsaxiom
9   {a>0}
10 end
11 {a>=0} // <- Auswahlregel

```

Hausübung

3.2 Qualitätsmanagement Standards

3.2.1 Softwarequalität

1. Was versteht man unter Softwarequalität? Was ist problematisch daran und wie steht der Begriff Softwarezwecktauglichkeit damit in Zusammenhang? (2 Punkte)

Lösungsvorschlag

Softwarequalität bezieht sich auf (funktionale und nicht-funktionale) Anforderungen. Dabei bedeutet Qualität vereinfacht, dass das Produkt seine Spezifikation erfüllen sollte. Für Software-Systeme ist dies allerdings problematisch, da es Qualitätsanforderungen von unterschiedlichen Beteiligten (Kunden, Entwickler) gibt, die sich z.T. widersprechen. Auch sind Softwarespezifikationen oft unvollständig und zum Teil widersprüchlich.

Statt nun die Konformität der Software mit ihrer Spezifikation zu betrachten, könnte man den Fokus auf ihre Zwecktauglichkeit legen. D.h. die Aspekte Zuverlässigkeit im Betrieb, akzeptable Leistungsfähigkeit für den vorgesehenen Gebrauch, Nutzbarkeit und gute Strukturierung und leichte Verständlichkeit betrachten.
(siehe T1.0 F6-9)

2. Worin besteht der Unterschied zwischen inneren und äußeren Qualitätsmerkmalen? (2 Punkte)

Lösungsvorschlag

Äußere Qualitätsmerkmale sind für den Benutzer des Systems wichtig, innere sind für das Wartungspersonal des Systems wichtig. Für den Auftraggeber sind beide Qualitätsmerkmale wichtig, da sich beide auf die operationalen Kosten auswirken.

3. Nennen und beschreiben Sie die in der Vorlesung besprochenen inneren und äußeren Qualitätsmerkmale nach ISO 9126. (4 Punkte)

Lösungsvorschlag

Änderbarkeit (I) : Merkmale, die sich auf den Aufwand für die Durchführung vorgegebener Änderungen beziehen.

Benutzbarkeit (A) : Merkmale, die sich auf den zur Benutzung erforderlichen Aufwand beziehen und auf die individuelle Bewertung einer solchen Benutzung durch eine festgelegte oder vorausgesetzte Gruppe von Benutzern.

Effizienz (A) : Merkmale, die sich auf das Verhältnis zwischen dem Leistungsniveau der Software und dem Umfang der eingesetzten Betriebsmittel unter festgelegten Bedingungen beziehen.

Funktionalität (A) : Vorhandensein von Funktionen mit festgelegten Eigenschaften.

Diese Funktionen erfüllen die festgelegten oder vorausgesetzten Anforderungen.

Übertragbarkeit (I) : Merkmale, die sich auf die Eignung der Software beziehen, von einer Umgebung in eine andere übertragen zu werden.

Zuverlässigkeit (A) : Merkmale, die sich auf die Fähigkeit der Software beziehen, ihr Leistungsniveau unter festgelegten Bedingungen über einen festgelegten Zeitraum zu bewahren.(siehe T1.0 F14-21)

4. Geben Sie zwei Beispiele für Qualitätsmerkmale, die dabei miteinander in Konflikt stehen können. Begründen Sie den Konflikt. (2 Punkte)

Lösungsvorschlag

z.B.

- Robustheit $\Leftrightarrow$ Performance
- Sicherheit $\Leftrightarrow$ Benutzbarkeit
- Sicherheit $\Leftrightarrow$ Effizienz
- Anpassungsfähigkeit $\Leftrightarrow$ Wiederverwendbarkeit
- ...

3.3 Softwareverifikation

3.3.1 Grundlagen Testen (5 Punkte)

1. Erläutern Sie ausführlich die Auswirkungen des Testens auf die Qualität.

Lösungsvorschlag

Testen erhöht indirekt die Qualität, da Fehler vor Auslieferung erkannt und beseitigt werden können. Auch wird die Prozessqualität verbessert, da Fehler dokumentiert, analysiert und damit Fehlhandlungen in Zukunft vermieden werden können. Zusätzlich wird das Vertrauen erhöht, wenn wenige oder keine Fehler gefunden werden.

2. Was ist der Unterschied zwischen einem Mangel und einem Fehler? In welchem Zusammenhang stehen die beiden zu Validierung und Verifizierung?

Lösungsvorschlag

Ein Fehler ist die Nichterfüllung einer Anforderung z.B. der Spezifikation, das heisst ein Abweichen des IST-Verhaltens vom SOLL-Verhalten. Ein Mangel liegt vor, wenn eine gestellte Anforderung oder eine berechnete Erwartung in Bezug auf den beabsichtigten Gebrauch der Software nicht erfüllt wird. Verifizierung prüft ob das System richtig realisiert wurde, d.h. auf Abwesenheit von Fehlern, Validierung prüft ob das richtige System realisiert wurde, d.h. die Abwesenheit von Mängeln.

3. Erläutern Sie den Unterschied zwischen statischem und dynamischem Test.

Lösungsvorschlag

Statische Tests führen den Code nicht aus, sondern überprüfen ihn allein anhand statischen Eigenschaften und Spezifikation. Hier muss der Code im ganzen vorliegen. Dynamische Tests führen den Code aus und testen ihn. Sei es durch spezifische Methodenaufrufe oder durch Ansprechen von Interfaces (GUI, API, ...). Der Code muss dazu nicht zwangsweise komplett vorliegen bzw. der Testende überhaupt Wissen über den Aufbau des Codes haben.

3.3.2 Hoare Kalkül (15 Punkte)

Betrachten Sie folgendes Programm S in Pseudocode:

```
1 function f(n) :  
2  
3   x := 1;  
4   y := 0;  
5       while (x <= n) do  
6           y := y + x;  
7           x := x + 1;  
8       end  
9       f := y;  
10 end
```

1. Beweisen Sie mittels Hoare Kalkül: $\{n \geq 0\}\{S\}\{f(n) = n * (n + 1)/2\}$
Nehmen Sie an, dass als Invariante gilt: $y = x * (x - 1)/2$
Geben Sie für alle Schritte die passenden Hoare Regeln an.

Lösungsvorschlag

```

1
2 {n>0}
3
4   {n>0}
5 2) x := 1 // -> Zuweisungsaxiom
6   {x<=n}
7
8   {x<=n}
9 2) y := 0 // -> Zuweisungsaxiom
10  {y=x*(x-1)/2 /\ x<=n}
11
12 {y=x*(x-1)/2 /\ x<=n} // Iterationsregel 1
13 while ( x <= n ) do
14   {y=n*(n+1)/2 /\ x=n /\ y = (x+1)*x/2}
15 3) y := y + x ; // -> Zuweisungsaxiom
16   {y=n*(n+1)/2 /\ x=n /\ y = (x+1)*x/2}
17 2)
18   {y=n*(n+1)/2 /\ x=n /\ y = (x+1)*x/2}
19 3) x := x + 1 ; // -> Zuweisungsaxiom
20   {y=n*(n+1)/2 /\ x=n+1 /\ y=x*(x-1)/2}
21 end
22 {x=n+1 /\ x= n*(n+1)/2} // Iterationsregel 1)
23
24 {x=n+1 /\ x= n*(n+1)/2}
25 2) f := y // -> Zuweisungsaxiom
26   {f(n) = n * (n + 1)/2}
27
28 {f(n) = n * (n + 1)/2}
29
30 2), 3) Kompositionsregel

```